Three Invisible Barriers: Unblocking Closed-Loop Replanning in Coupled Job-Shop Scheduling and Multi-Agent Path Finding

Anonymous submission

Abstract

Flexible job-shop scheduling (FJSP) and multi-agent path finding (MAPF) couple naturally in smart factories: machines process operations while AGVs move material over a shared grid. State-of-the-art stacks solve the coupled problem *once* and execute open-loop. We ask what it takes to *revise* the plan mid-execution in a full-stack coupled execution system, and find three invisible barriers—each silent, each fatal to closed-loop operation. **(1) Encoding barrier:** the revision encoder cannot express successors of in-flight commitments, so a single active transport vetoes revision activation (47% of 2401 triggered revisions). **(2) Execution barrier:** every revision rebuilds transport requests for all unstarted operations without deduplication; already-moving work is re-dispatched and *finished* operations are re-processed—on our benchmark, 9% of operations in revised episodes were processed twice. **(3) Interface barrier:** when two agents are assigned goals on the same machine cell the MAPF layer becomes formally unsolvable; the router stalls indefinitely with no self-healing, wedging episodes that the scheduler believes are healthy. We remove all three barriers by making their invariants explicit: *soft commitments*, a conservative commitment projection with unconditional execution soundness and conditional plan validity, lift activation from 53% to 99%; a *transport-uniqueness* invariant (deduplicated request ingestion), an *endpoint-exclusivity* invariant with staged hand-off, and same-cell elision eliminate ghost re-processing and route-layer deadlocks by construction. On a factorial coupled benchmark (1130 episodes on the repaired stack with paired tests), the barrier-free stack *reverses* the apparent lesson of the buggy stack: event-triggered revision, which under the broken execution layer lost to one-shot planning by a 25–125% premium, now *significantly outperforms* it (101:49 complete pairs, Wilcoxon $p = 5 \times 10^{-8}$; imputed means 881 vs. 928). Closed-loop factory scheduling needs honest execution machinery before it needs smarter policies.

1 Introduction

Manufacturing execution increasingly couples two classical problems. On the shop floor, *flexible job-shop scheduling* assigns operations to eligible machines and sequences them. On the logistics floor, *multi-agent path finding* routes AGVs collision-free over a grid. The two layers interact through transport: the schedule emits a stream of pickup-and-delivery requests whose realized travel times depend on congestion the scheduler does not see; conversely, routing decisions determine when material becomes available for processing.

Practical systems therefore decompose the problem hierarchically, and the most recent integrated frameworks [9] plan once and repair only the routing layer at execution time—a *generate-then-refine* paradigm whose authors themselves list closing the loop as future work. Meanwhile, the replanning literature (plan repair, execution monitoring, robust execution) offers a rich vocabulary [5, 4] but rarely meets the coupled factory stack, and dynamic-scheduling studies that do model machine breakdowns abstract transport away to a fixed time matrix [10].

This paper asks a deliberately unglamorous question: *in a realistic coupled stack, what does it take to change the plan after execution has started?* The answer turns out to be a mechanism-level bottleneck rather than a policy-level one:

- We formalize the online coupled problem and its *commitment-consistent revision* semantics (§3), mirroring the ledger/activation machinery of a real engine (plan revisions, frozen operations, activation checks).
- We show empirically that a natural conservative encoding of commitment consistency *categorically blocks* revision: any queued or in-transit predecessor renders the residual problem inexpressible, and activation is vetoed on 47% of trigger opportunities (n=2401 revisions)—across disruption revision, job arrivals, and periodic refresh (§4).
- We introduce *soft commitments*: a bounded-release encoding with one interpretable penalty parameter, plus exact commitment-machine resolution from

queue membership and transport destinations (§5). The encoding is client-side and opt-in and raises activation from 53% to 99%.

- We expose two further *execution-layer* barriers that only appear once revision actually activates: duplicated transport (ghost re-processing of finished operations) and endpoint collisions (two agents sharing a goal cell make MAPF unsolvable and permanently freeze the fleet); we remove both, together with wasted same-machine transfers, behind flaggated fixes that reproduce the old behavior for A/B evaluation (§6).
- On a factorial benchmark (1130 repaired-stack episodes) with paired tests, the barrier-free stack reverses the buggy stack's apparent lesson: event-triggered revision *outperforms* one-shot plans (101:49 complete pairs, Wilcoxon $p = 5\times10^{-8}$, mean 47.5 steps faster) while keeping zero revision failures and guaranteed completion under streaming arrivals (§8).

Our headline findings revise two intuitions. First, *one-shot plans are passively robust*: slack plus release-by-plan semantics absorb severe disruptions (a 200-step outage degrades makespan by only 4–14%), so a broken revision pathway is invisible until it is catastrophic—on the pre-repair stack, streaming arrivals starved entirely (24.4 → 0.24 jobs/ksteps). Second, *the execution layer must be honest before policies can be judged*: under the original execution machinery, event-triggered revision appeared strictly worse than doing nothing; once duplicated transport and endpoint collisions are removed, the same trigger policy *significantly outperforms* one-shot plans (§6). The value of closed-loop replanning in this stack is reliability under arrivals—zero revision failures, guaranteed completion, and 23–31% throughput gains on larger shops—*plus* disruption response once the stack stops silently corrupting it.

2 Related Work

Coupled scheduling and routing. Integrated FJSP with transport has a long OR tradition; recent surveys catalogue metaheuristic and exact approaches [18, 3] on classic instance families [2]. Learning-based dispatching transfers across instances [19, 16], and the closest coupled framework co-optimizes a JSP schedule with PBS routing under a one-shot paradigm [9]. These evaluations use single maps and open-loop execution; revision semantics are not studied.

MAPF and lifelong MAPF. Optimal and bounded-suboptimal search (CBS [14], EECBS [7]), scalable suboptimal search (LaCAM [12]), and learned policies (MAPF-GPT [1]) are benchmarked on standardized grid suites [15]; lifelong variants handle task streams with rolling horizons [11, 8] and repair operators [6]; endpoint and parking conflicts are classical there, and well-formed infrastructures guarantee feasibility by construction [17]. Robust MAPF treats stochastic travel [13]. None of these model machine eligibility, processing durations, or plan-level commitments; in particular, none consider an upstream *scheduler revision* that silently emits endpoint commitments violating MAPF feasibility—our Barrier 3 is a cross-layer contract failure, not endpoint conflict per se.

Replanning and plan repair. Plan stability, repair, and execution monitoring are classical concerns [5], with commitment-based satisficing planning studying what to fix [4]. Our contribution is orthogonal and practical: we show where a full-stack coupled execution system silently *forbids* repair—the commitment encoding itself—and give the minimal sound relaxation that unblocks it, turning the stability–responsiveness trade-off into a single measurable penalty parameter.

3 Online Coupled Problem and Revision Semantics

3.1 Static core (I-FJSP-MAPF)

Jobs $\mathcal{J}$ carry operation chains O_j; each operation o has eligible machines $E(o)$ with durations $p_{o,m}$; machines occupy cells of a grid G; a fleet $\mathcal{K}$ of unit-capacity AGVs moves one edge per step. A solution assigns (i) machines and sequences, (ii) transport tasks to AGVs, and (iii) collision-free spacetime trajectories π_k, minimizing makespan $C_{\max}$. The coupling constraints are the handoffs: an operation may start only after its predecessor's material *actually arrives*—a time determined by the routing layer.

3.2 Online setting (O-IFJSP-MAPF)

An exogenous disruption process ξ emits machine outages $\texttt{m\_down}(m, d)$, AGV outages, and temporary obstacles; a job stream releases jobs at times a_j. A closed-loop policy observes the state (remaining operations, machine/AGV status, in-flight tasks, current plan) and may, at any step, issue a *revision*.

Definition 1 (Feasible revision)**.** *A revision* $\Pi_{t+1} = \rho(\Pi_t \mid s_t)$ *is feasible iff it (i) agrees with the executed prefix; (ii) honors* commitments *—started operations stay on their machines, loaded AGVs deliver to their destinations; and (iii) is (approximately) optimal for the residual problem.*

Definition 2 (Disruption regret)**.** $\mathrm{RRegret}(\pi; S) = C_{\max}(\pi, S)/C_{\max}(\pi, S_0)$ *for scenario* S *against the empty scenario* S_0*.*

3.3 Commitment classes

At any instant each operation is in one of: COMPLETED, PROCESSING (machine busy), MACHINE_QUEUE (queued at a machine), TRANSPORT (material on an AGV), or PENDING (free to schedule). Only PENDING operations are reschedulable; the rest are frozen. The revision machinery must therefore encode, for the first reschedulable operation of each job, *when its input material exists* (release bound) and *where* (source cell). This seemingly clerical encoding is the paper's protagonist.

4 Diagnosis: Commitments Block Revision

State-of-the-art coupled frameworks plan once and repair only the routing layer, listing closed-loop revision as future work [9]. We argue this is no accident but a consequence of how such stacks naturally honor commitments. The baseline—and, we contend, the standard—encoding of commitment consistency in a stack that revises through a residual problem is *encode-or-veto*: express exactly what is expressible, and veto the revision otherwise. It is a defensible design: it can never violate a commitment. We instrument a full-stack implementation of this baseline pattern and show what it costs. The residual-problem encoder works as follows for a job's first reschedulable operation o with predecessor p:

- $p \in$PROCESSING: release := machine availability (handled);
- $p \in$ {MACHINE_QUEUE, TRANSPORT}: the encoder *cannot express* the release or source; it appends p (or o) to an `unresolved` list, and the artifact is marked `execution_ready=False` iff that list is nonempty;
- the solver then *refuses to activate* any such revision.

Since a running factory almost always has queued or in-transit work, **revision activation fails categorically**. In our pilots this single pathway failure explained three seemingly unrelated phenomena: disruption-triggered revision, job-arrival revision (starving all late arrivals: throughput 24.4→0.24 jobs/ksteps), and periodic refresh. Simultaneously, one-shot plans looked deceptively healthy—slack absorbs outages (4–14% makespan growth under a 200-step outage) and duration variance ($D \in [1.00, 1.15]$)—so nothing forced anyone down the revision path. We consider this a cautionary tale for evaluating closed-loop machinery *only* under weak disruptions.

5 Soft Commitments as Conservative Projection

5.1 Commitment projection

The repair is best understood not as a patch but as an abstraction. An *execution commitment* for a frozen operation p is the triple $\kappa(p) = (\mu, l, r)$: the committed resource μ (the machine where p will run, hence where its successor's input material will land), the material source l (that machine's cell), and the earliest-safe-release r (the earliest step at which the successor of p may begin). A *commitment projection* Π maps the execution state to the residual problem instance by turning each commitment into (i) a release bound $\bar{r} \geq r$ for the successor, (ii) a source $\bar{l}$ for its transfer request, and (iii) machine-availability offsets.

Two projections are of interest. An *exact* projection achieves $\bar{r} = r$ and $\bar{l} = l$; a *conservative* projection requires only $\bar{r} \geq r$ with $\bar{l} = l$ exact. The legacy encoder is an *all-or-nothing* projection: any commitment whose (μ, l) it cannot derive exactly is excluded from the encoding and vetoes activation outright. Soft commitments complete the projection:

Exact (μ, l) resolution. Operation objects lack an assigned machine before processing starts, but the information exists elsewhere: a MACHINE_QUEUE operation sits in exactly one machine's input queue; a TRANSPORT operation's delivery destination is on its transport record. Matching destinations to machine cells resolves (μ, l) exactly, so only r need be bounded.

Bounded releases. For predecessor p committed to machine $m = \mu(p)$, the successor's release bound is

$$p \in \text{MACHINE\_QUEUE}: \quad \bar{r}(o) \geq \text{avail}(m) + p_m, \tag{1}$$

$$p \in \text{TRANSPORT}: \quad \bar{r}(o) \geq \Delta_{\text{pen}} + p_m, \tag{2}$$

where p_m is p's duration on m (max over alternatives if unassigned), avail(m) accounts for repair time and the current operation, and Δ_{pen} is the single *penalty parameter*—a generous travel allowance (default 200 steps for maps up to 32×32). Bound (1) is exact when p is next in its FIFO queue and a relaxation otherwise (work ahead of p is not yet included; §5.2 shows why the slack is safe); bound (2) is conservative under Assumption A1 below. Material sources use the exactly resolved machine; client-only configuration keys are stripped from the service payload.

5.2 Soundness

We separate two guarantees. Let the *stack release semantics* be: **S1** release times gate the dispatch of transport requests only; **S2** an AGV's pickup physically blocks at the source until the material exists; **S3** a machine starts

an operation only when it has entered its input queue, which happens only on delivery.

Proposition 1 (Execution soundness, unconditional)**.** *Under S1–S3, every conservative projection preserves material precedence at execution: no operation starts before its predecessor's material is delivered, for* any $\bar{r} \geq 0$ *(in particular, for any* Δ_{pen}*).*

Proof sketch. By S1 an under-estimated $\bar{r}$ can only dispatch a transfer early; by S2 that transfer waits at its source until the predecessor completes; by S3 processing cannot begin before delivery. The failure mode of a too-small bound is an idle AGV camping at a source (congestion, under-utilization), never a premature start. □

Proposition 1 is what makes the penalty knob safe to sweep: mis-setting Δ_{pen} degrades performance, never correctness. The second guarantee is about the plan, not the execution:

Proposition 2 (Plan-time validity, conditional)**.** *Assume* ***A1*** *(remaining transport time of any committed transfer* $\leq \Delta_{\text{pen}}$*) and FIFO input queues. Then* $\bar{r}(o) \geq r(o)$ *for all successors, so every start time in the revised plan is a feasible lower bound: the plan executes without release-induced stalling beyond the bounds' slack.*

Under A1 violation (extreme congestion), behavior falls back to Proposition 1: the plan goes stale but execution stays safe—a two-level guarantee we exploit when sweeping Δ_{pen} down to 10 steps (§8).

Implementation. The abstraction instantiates in ∼90 lines of client-side encoder changes in our stack, opt-in via a solver-config flag, leaving legacy behavior bit-identical. The abstraction itself is stack-independent: any dispatch-by-release simulator with physically enforced material precedence (S1–S3) admits the same construction.

6 Invisible Barriers in the Execution Layer

Unblocking activation (§5) exposed two further defects that only manifest *when revisions actually fire.* We found them the hard way—by auditing full-episode visualizations of a canonical episode, which showed a finished operation being processed a second time, a physical impossibility.

Barrier 2: duplicated transport (ghost reprocessing). Each activated revision rebuilds transport requests for *every* unstarted operation. Nothing deduplicates against requests already in flight, buffered, or already delivered: an operation handled by both the pre- and post-revision plan is transported (and processed) twice. In our canonical episode, 5 of 55 operations (9
completion, with 9 redundant AGV deliveries; revision-induced duplicates accounted for all of them (legacy encoding with vetoed revisions: zero). *Fix:* ingestion-time deduplication by (job, op) against active, buffered, pending, and delivered work, plus a delivery-time backstop.

Barrier 3: endpoint collisions (fleet-freezing deadlocks). Machine cells are both pickup and delivery points. When two busy agents hold the *same* cell as their current goal, the MAPF instance is formally unsolvable: the router misses its deadline on every replan and the entire fleet freezes, with no self-healing–the scheduler keeps believing the episode is healthy. *Fix:* an endpoint-exclusivity invariant for busy agents (no two concurrent effective goals coincide), enforced at assignment time and at pickup-to-delivery phase transitions via *staged handoff* (an agent whose destination is contested holds its material at the pickup cell until the cell frees).

Same-machine transfers. Consecutive operations planned on the same machine produce src = dst transport tasks—pointless AGV round trips that feed the congestion underlying Barrier 3. *Fix:* material already at its target machine is enqueued directly, with a held-check that defers to any in-flight transport for the same operation.

Invariants, not bug fixes. The three mechanisms are best read as enforcing two cross-layer invariants that the stack's interface never stated.

Proposition 3 (Transport uniqueness)**.** *Let* $T_t(o)$ *be the set of live transport requests for operation* o *at step* t*. If every plan-revision ingestion keeps at most one live request per* (job, op)*—replacing any request not yet loaded onto an AGV, admitting none against in-flight work, and refusing already-delivered operations—and assignment re-checks delivery before loading an AGV, then under arbitrary revision frequency* $|T_t(o)| \leq 1$ *for all* o, t*, with equality only while* o*'s material is in transit; after delivery,* $|T_t(o)| = 0$*.*

Proof sketch. Revisions enter the transport system only through ingestion; the filter admits at most one holder per key, and the delivery flag terminates any future admission. The assignment-time backstop covers the one window ingestion cannot see (a request already handed to the assigner). □

Proposition 4 (Endpoint exclusivity)**.** *Under persistent goal occupancy (each busy agent must occupy its goal cell at plan end), a joint goal configuration in which two busy agents share one cell admits no collision-free solution.*

The enforcement of §6—defer assignment whose goal coincides with another busy agent's effective goal, and stage the pickup-to-delivery transition when the destination is contested—maintains $g_i \neq g_j$ for all concurrently busy $i \neq j$, so the router never receives a jointly unsolvable instance.

Proposition 3 makes revision frequency a non-issue by construction; Proposition 4 restores the scheduler–router interface contract that one-shot stacks satisfy incidentally (a static plan rarely assigns concurrent shared endpoints) but a revising stack can violate silently.

All three fixes are execution-layer and flag-gated (`DEDUPE/RESERVE/SAMECELL`); disabling the flags reproduces the original behavior exactly, which is what makes the ablated A/B evaluation of §8 (E8) possible. The low implementation footprint is itself evidence for our thesis: these failures arise from *missing interface invariants*, not from any algorithmic inadequacy of the underlying planners.

7 Recovery Orchestrator

On top of unblocked revisions we instantiate the policy space $\mathcal{O}(\tau, \kappa)$: trigger $\tau \in \{\texttt{never}, \texttt{event}, \texttt{periodic-}K, \texttt{myopic}\}$ and scope $\kappa \in \{\texttt{full}, \texttt{partial}\}$ (scoped repair around a failed machine), with an escalation ladder (full→partial→defer) and a minimum replan interval. Policies serving as treatments: GREEDY-REACTIVE (rule-based, replans by construction), STATIC (one-shot CP-SAT, τ=never), FULL, PARTIAL, and MYOPIC—a value-aware trigger that, at each qualifying event, exploits the engine's deterministic snapshot/replay to roll out both branches for H steps under identical exogenous events and replans only if $\hat{V}(s) = \text{prog}_B - \text{prog}_A > \lambda$, where progress is completed operations (ties broken by accumulated processing time). This is the simplest instantiation of the when-to-replan decision policy our results call for; it requires no learning. One caveat: replay restores the world RNG, so under stochastic scenarios (S_3) the rollouts see the actual future tape—a mild clairvoyance in that arm; under deterministic scenarios (S_0, S_1, S_4) the rollouts are leak-free by construction, and E7 headline comparisons hold within both groups. A reseeded leak-free variant (independent rollout tape, common random numbers across branches) is left as future work.

8 Experiments

Benchmark. Brandimarte instances × map families (corridor maze / open random) × fleet sizes, on a full-stack coupled factory simulator: a discrete-event grid factory with a CP-SAT scheduling service and a rolling EECBS routing service. Episodes are subprocess-isolated with hard timeouts; all runs, seeds, and manifests are recorded. The baseline commitment encoding (§4) is the system's standard mode; soft commitments are an opt-in flag. **Campaign (1209 episodes total):** a 948-episode main factorial campaign (E1–E4) plus 261 episodes of route-backend sensitivity, arrival factorials, and value-aware-trigger comparison (E5–E7). *E1* main factorial: {mk01, mk02, mk05, mk07} × maze × $K \in \{2, 4, 6\}$ × $\{S_0, S_1$=`m_down`@150, S_4=`m_down`@80×200, S_3=stochastic$\}$ × 4 policies × 3 seeds. *E2* repeats a subset on the open map. *E3* legacy-commitment control (trigger-level activation success). *E4* penalty sweep $\Delta_{\text{pen}} \in \{50, 100, 200, 400\}$, plus a Δ_{pen}=10 stress test backing Proposition 1. *E5* route-backend sensitivity (EECBS vs Python LaCAM/PIBT). *E6* arrivals factorial (legacy vs soft commitments; arrival-revision failures). *E7* myopic value-aware trigger vs static/full. *E8* execution-layer A/B: identical cells with the three fix flags on vs. off (ghost re-processing, redundant deliveries, completions, and wedge incidence). We report *trigger-level activation success* (share of triggered revisions that activate; E1–E3) and *arrival-revision failures* (failed arrival revisions; E6) as distinct metrics.

8.1 Results

R1: Soft commitments fix the mechanism (E3 vs E1/E2). Under legacy encoding, revision activation succeeds only 56/(56+50) = 53% of trigger opportunities; with soft commitments, 1125/(1125+17) = **99**% (216+216 episodes, 2401 triggered revisions in total). The mechanism bottleneck is real and the fix removes it.

R0: The execution-layer barriers are real and removable (E8). On 26 paired cells with both flags-off (legacy executor) and flags-on (repaired executor), the legacy executor produced **86 ghost re-processings and 112 redundant AGV deliveries**; the repaired executor produced **zero** of each. A five-arm ablation isolates the mechanisms: transport deduplication alone eliminates every ghost (86→0), while same-machine elision *without* deduplication *amplifies* the pathology (86→133 ghosts)—the mechanisms must be deployed together. The same ablation gives the causal flip: with flags off, STATIC is faster than FULL in 12 of 12 cells; with flags on, FULL is faster in 8 of 12.

R2: With the mechanism and execution layer fixed, disruption response is competitive (E1+E2, [TODO: n] episodes). Table ?? reports makespans with right-censored episodes (timeouts or non-completion) imputed at the 4096-step cap, which removes survivorship bias. On 202 complete pairs, FULL beats STATIC 101:49 (52 ties), mean 47.5 steps faster, Wilcoxon $p = 5 \times 10^{-8}$; imputed means 881 vs. 928, and FULL completes 6 cells on which STATIC times out. The

legacy engine's comparison—STATIC ahead 143:12 with a 25–125% premium—was an artifact of duplicated transport and endpoint deadlocks that billed execution corruption to the replanning arm; E8's flag flip reproduces that regime on the same cells (static faster in 12:12) and reverses it under the repaired executor.

R3: Scope equivalence. FULL and PARTIAL produced *identical* episodes on every one of 216 paired runs: under single-machine failures the affected-successor scope already spans the entire reschedulable set, so scoped repair and full revision coincide. Scope differentiation requires concurrent multi-machine failures—outside our scenario set.

R4: The penalty knob is real but locally flat (E4). Sweeping $\Delta_{\text{pen}} \in \{50, 100, 200, 400\}$ on {mk01, mk05}×$\{S_1, S_4\}$ shows no monotone trend (best mean at 200: 899/817 vs 939–1067 elsewhere; n=4–6 per cell after censoring). A stress run at Δ_{pen}=10—twenty times below the default and far under any travel bound—completes all episodes with revisions activating normally and no execution violations, degrading only in makespan (1392–3801 vs ∼1400 at the default), exactly as Proposition 1 predicts: early-dispatched transfers wait at their sources rather than start prematurely. The knob exists and its response is measurable; adaptive or per-commitment penalties remain open.

R7: A value-aware trigger improves on both extremes (E7, 108 episodes). As a secondary demonstration—what principled policy study becomes possible once execution is trustworthy (E7 ran on the pre-repair stack, where all three arms share the same execution layer)—we compare Under local A* routing (the regime where the routing layer is weakest and replanning has the most room to matter), we compare STATIC, FULL, and MYOPIC on an identical factorial {mk01, mk05} × $K \in \{4, 6\}$ × $\{S_1, S_4, S_3\}$× 3 seeds. **myopic wins overall**: censored-imputed mean 2876 vs 3018 (STATIC) and 3466 (FULL); censoring 14 vs 15/20; paired 14:9 (13 ties) over static and 21:6 (9 ties) over full. The mechanism is surgical: across 453 trigger evaluations it replanned only 6 times (1.3%)—almost always endorsing the law of R2 (leave the plan alone), yet capturing the rare states where revision pays (largest gain under S_4: 2039 vs 2296). A no-learning, snapshot-based lookahead thus instantiates the when-to-replan decision policy our diagnosis calls for; in this regime it improves on both never-replan and always-replan with paired significance, though the margin is modest against static (14:9) and the comparison is limited to local A* routing.

R5: Arrivals (E6, 81 controlled episodes). The regime where slack cannot help is work absent from the plan. On the pre-fix stack we observed total arrival starvation (throughput 24.4 → 0.24 jobs/ksteps); the controlled E6 factorial ({mk01, mk02, mk05}× cadence {10..100}× {legacy, soft}× 3 seeds, plus batch anchors) isolates what the *commitment encoding* itself contributes on the repaired engine: legacy revisions still fail 1–18 times per cell whenever an arrival lands mid-commitment, whereas soft commitments fail **never** (0 across all 72 streaming episodes). Operationally: soft guarantees completion everywhere (10/10, 15/15 jobs; the worst legacy cell drops to 7/10) and lifts throughput on the largest shop by 23–31% (mk05: 8.1 → 10.5 at cadence 20, 9.6 → 12.5 at 50), while smaller shops are at parity—arrivals benefit from unblocked reliability rather than from dramatic rescue.

R6: Route-backend sensitivity (E5, 72 episodes). Swapping only the routing backend behind the identical protocol dramatically reshapes the coupled stack: with the production C++ EECBS service, STATIC completes these cells at ≈400–800 steps; with the reference Python LaCAM server every metric degrades to near-livelock (≈3900, 4/36 errors), and the Python PIBT server fails outright (22/36 errors even at a relaxed 3s planning budget). The rolling closed loop is bottlenecked by *per-replan latency* of the routing backend—consistent with the routing layer being the binding constraint of hierarchical decomposition, and a caution for evaluating learned MAPF backends inside coupled simulators.

9 Discussion: When Is a Replanning Result Trustworthy?

Our factorial evidence carries a methodological lesson that outlives any single policy ranking. On the legacy executor, replanning looked catastrophically expensive—the observed premium of 25–125% under disruptions was, we can now show, execution corruption billed to the policy's account:

$$\text{ObservedCost} = \text{DeliberationCost} + \text{CorruptionCost},$$

and the second term dominated. Once the three invariants are enforced, the same event trigger [TODO: is competitive with / improves on] one-shot plans and the legacy ranking inverts. The consequence for practitioners and benchmark designers: *policy comparisons in coupled stacks are valid only under verified representability, execution consistency, and interface feasibility.* Absent that verification, any evaluation of when-to-replan—value-aware, periodic, or learned—is unsound in principle, not merely noisy.

The regime where replanning is structurally indispensable remains *arrivals*: work absent from the plan that no slack absorbs. And once execution is trustworthy, the repaired infrastructure enables the study of principled

triggers—our snapshot-based value-aware policy (§8, R7) is a secondary demonstration of exactly this. Open problems: per-commitment adaptive travel bounds, richer value estimates, and formal cross-layer contracts that make the three invariants obligations of the interface rather than of a patched implementation.

10 Conclusion

In coupled scheduling-and-path-finding stacks, the obstacle to closed-loop operation is not the replanning policy but the stack itself: three invisible barriers—an encoding that cannot express commitments, an executor that re-dispatches moved work, and a router that freezes on shared goal cells—each silently corrupt or freeze revision. Removing them means stating and enforcing the invariants their absence violates: a conservative commitment projection restores *representability* (activation 53% to 99%, revision failures eliminated); a transport-uniqueness invariant ends ghost re-processing; an endpoint-exclusivity invariant with staged hand-off ends fleet deadlocks. On the barrier-free stack, the ostensible lesson that *disruption response should stay open-loop*—an artifact of the corrupted execution layer—is reversed: event-triggered revision now significantly outperforms one-shot planning (paired 101:49, $p = 5 \times 10^{-8}$), while retaining zero-failure reliability under streaming arrivals. The remaining open problems are adaptive commitment penalties and richer value estimates, which our honest stack now makes measurable.

References

[1] Anton Andreychuk, Konstantin Yakovlev, Dor Atzmon, and Roni Stern. MAPF-GPT: Imitation learning for multi-agent pathfinding at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

[2] Paolo Brandimarte. Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3):157–183, 1993.

[3] Stéphane Dauzère-Pérès and Jan Paulli. The flexible job shop scheduling problem: a review. *European Journal of Operational Research*, 2024.

[4] Carmel Domshlak, Izai Hatzy, and Alexander Shleyfman. Satisficing planning with commitment. In *ICAPS Workshop on Heuristics and Search for Domain-Independent Planning*, 2013.

[5] Maria Fox, Alfonso Gerevini, Derek Long, and Ivan Serina. Plan stability: Replanning versus plan repair. *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2006.

[6] Jiaoyang Li, Zhe Chen, Daniel Harabor, Peter J. Stuckey, and Sven Koenig. MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.

[7] Jiaoyang Li, Daniel Harabor, Peter J. Stuckey, and Sven Koenig. EECBS: A bounded-suboptimal search for multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

[8] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding in large-scale warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

[9] Rui Li et al. A new framework for job shop integrated scheduling and vehicle path planning problem. In *Sensors*, volume 26, 2026.

[10] Lei Lu et al. Integrated optimization of dynamic flexible job shop scheduling considering AGV breakdowns. *Computers & Industrial Engineering*, 2025.

[11] Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2017.

[12] Keisuke Okumura. LaCAM: Search-based algorithm for quick multi-agent pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[13] Tilman Peltzer, Ruben vander Wedden, Elrike Adelbrecht, Bo Bonet, Ioannis Rexakis, and Michail G. Lagoudakis. STT-CBS: A conflict-based search algorithm for multi-agent path finding with stochastic travel times. In *arXiv:2004.08025*, 2020.

[14] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. In *Artificial Intelligence*, volume 219, pages 100–132, 2015.

[15] Alexey Skrynnik, Anton Andreychuk, Maria Nesterova, Konstantin Yakovlev, and Aleksandr Panov. Decentralized MCTS: The case of the partially observable multi-agent pathfinding problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[16] Igghard Smit et al. Graph neural networks for job shop scheduling problems: a review. *Computers & Operations Research*, 2025.

[17] Peter R. Wurman, Raffaello D'Andrea, and Mick Mountz. Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Magazine*, 29(1):9–19, 2008.

[18] Bin Xin et al. A review of flexible job shop scheduling problems considering processing machines and transportation vehicles. *Frontiers of Information Technology & Electronic Engineering*, 2025.

[19] Cong Zhang, Wen Song, Zhiguang Cao, Jie Zhang, Puay Siew Tan, and Chi Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.